

WARFRAME JARVIS

Engineering Accomplishments Log

Patrick Onuoha Jr. · DS/DE · Denton, TX

Updated: June 2026 · Stack: Supabase · FastAPI · React · Python ETL

This is a living document. It tracks what Patrick has built, learned, and accomplished while developing Warframe Jarvis — a personal Warframe analytics and build intelligence platform. Patrick is a DS/DE, not a SWE. Everything here was built from scratch in territory that was not his home turf.

I. Architecture & Systems Design

Database Design

Built and evolved a multi-schema Warframe database across two backends.

- ✓ SQL Server prototype (Phase 0 origin)
- ✓ Migrated fully to Supabase Postgres
- ✓ Base vs User schema separation (wf_base / wf_user)
- ✓ Warframe, Weapon, Arcane, Companion entity modeling
- ✓ Archon Shard relational structure with Tauforged scaling
- ✓ Variant / Apex Variant Constitution tracking
- ✓ Incarnon Adapter boolean architecture (one weapon row, no duplication)
- ✓ Melee arcane slot added to weapons table

ETL & Seeding

- ✓ 129 arcanes seeded via arcane_seed.sql (canonical truth)
- ✓ MODERN_ADDENDUM system for new arcanes without touching canonical seed
- ✓ 634-weapon database seeded from WFCD data with normalized Incarnon architecture
- ✓ 57 Warframe loadouts migrated from Excel workbook (Stress_Test_F1.xlsx)
- ✓ KPM session data migrated from Excel to Supabase

II. API & Backend Integration

Designed and implemented systems that communicate between Supabase, FastAPI, and React.

- ✓ Save/load workflows across all Arsenal fields

- ✓ Data persistence for arcanes, weapons, melee arcane, Incarnon state
- ✓ CRUD operations (create, read, update, delete) on build data
- ✓ State synchronization between frontend components and DB

Concepts encountered for the first time:

- ✓ Client/server architecture
- ✓ Data flow patterns
- ✓ API consumption from a React frontend
- ✓ Backend integration without a backend engineering background

III. Frontend Development

Built a full React UI with no formal frontend experience prior to this project.

Components Built

- ✓ Frame cards with cultivation color integration
- ✓ Arsenal editor modal (fully tabbed: Arsenal / Archon Shards)
- ✓ Archon Shard editor (ShardEditModal.jsx — refactored from ~880 to ~308 lines)
- ✓ Incarnon Adapter chip UI (OFF / ACTIVE states with gold border + badge)
- ✓ Melee arcane slot with save/load
- ✓ Search / autocomplete (unified across weapons, arcanes, melee arcanes)
- ✓ Copy Arcane Setup dropdown
- ✓ Clear All for current and goal shard loadouts
- ✓ Dynamic badges (Variant Constitution, Apex Variant Constitution)
- ✓ Navigation systems

Concepts encountered for the first time:

- ✓ React components, props, state
- ✓ Conditional rendering
- ✓ Reusable component architecture
- ✓ Modal systems
- ✓ Controlled inputs and autocomplete patterns

IV. Search & Filtering Systems

- ✓ Weapon autocomplete with ranked results
- ✓ Arcane autocomplete
- ✓ Category-aware arcane filtering (Warframe / Primary / Secondary / Melee slots)

Concepts encountered for the first time:

- ✓ Dynamic filtering

- ✓ Search ranking
- ✓ UX optimization for data categorization

V. Taxonomy & Information Architecture

One of the most underrated accomplishments. Patrick designed a full classification system:

- ✓ 14 Cultivation Schools with wuxia dao names and Chinese characters
- ✓ 57 Warframe dao assignments with lore justification
- ✓ Doctrine categories for build philosophy tagging
- ✓ Variant Constitution naming system (Variant / Apex Variant)
- ✓ Build status tags and tier assignments (S / A / B / C)
- ✓ Hex color system per frame tied to cultivation lore

Concepts encountered for the first time:

- ✓ Classification system design
- ✓ Metadata architecture
- ✓ Domain modeling
- ✓ Information hierarchy

VI. Product Design & Prioritization

Patrick is not coding random features. Every build decision reflects intentional product thinking.

- ✓ Homepage vs. collection vs. loadout vs. research — deliberate placement
- ✓ Feature prioritization across a multi-phase roadmap
- ✓ Scope discipline (rejecting feature creep mid-session repeatedly)
- ✓ User journey design for a solo founder-user product

Concepts encountered for the first time:

- ✓ Product thinking as a non-PM
- ✓ Roadmap architecture
- ✓ Feature vs. foundation tradeoffs

VII. Version Control

- ✓ GitHub repository setup and management
- ✓ Commit / push / pull workflows
- ✓ Multi-machine workflow (desktop ↔ laptop synchronization)
- ✓ PAT token management (encountered expired token issue, resolved)

Concepts encountered for the first time:

- ✓ Version control fundamentals
- ✓ Branch safety mindset
- ✓ Source control as professional discipline

VIII. Debugging

Participated in debugging real production issues:

- ✓ Hover color conflicts
- ✓ Overlay rendering issues
- ✓ Missing or malformed data edge cases
- ✓ Arcane filtering slot mismatch
- ✓ Incarnon toggle UI state management

Concepts encountered for the first time:

- ✓ Root cause analysis
- ✓ Incremental testing
- ✓ Safe deployment mindset (don't touch what isn't broken)

IX. Prompt Engineering

This may be Patrick's strongest emerging technical skill outside DS/DE.

- ✓ Defining requirements before implementation
- ✓ Scoping features before writing code
- ✓ Rejecting scope creep in real time
- ✓ Translating vision into implementation-ready instructions
- ✓ Coordinating multiple AI systems (Claude as PM, ChatGPT as implementer)
- ✓ Managing context across long multi-session builds

Concepts encountered for the first time:

- ✓ Requirements engineering
- ✓ Technical communication as a discipline
- ✓ AI orchestration across tools

X. Data Engineering & Data Quality

Patrick's DS/DE background came forward during these sessions — this is home turf applied to a personal project.

Production Data Quality Audit

Performed first production data quality audit on the live arcane dataset.

- ✓ Traced full ETL pipeline: arcane_seed.sql → seed_arcanes.py → Supabase
- ✓ Identified taxonomy mismatch — 'Other' category flagged as incorrect classification
- ✓ Identified count discrepancy in Zaw arcanes
- ✓ Built completeness audit across all arcane categories
- ✓ Quantified missing records by domain before attempting remediation
- ✓ Removed invalid arcane records from production
- ✓ Verified and locked arcane count at 129
- ✓ Created Arcane_Data_Audit.md as a living data-quality document

Skills exercised:

- ✓ Data lineage analysis
- ✓ ETL investigation
- ✓ Data quality auditing
- ✓ Taxonomy validation
- ✓ Production dataset verification

As a previous Claude session noted: this is straight-up Data Engineering work, not frontend work.

XI. Repository & Documentation Architecture

Reorganized Jarvis from a working prototype into a professional multi-layer project structure.

- ✓ Separated concerns into dedicated folders: API, Architecture, Assets, Audits, DB, Docs, ETL, Planning, Frontend
- ✓ Created dedicated folders for documentation, audits, cultivation SQL, and database guides
- ✓ Established scalable documentation structure for future development
- ✓ Established permanent documentation standards
- ✓ Established audit documentation standards
- ✓ Established session logging standards

Concepts encountered for the first time:

- ✓ Project repository architecture for multi-layer applications
- ✓ Separating concerns at the file system level
- ✓ Documentation as a first-class engineering artifact

XII. Arcane Collection System — Foundation

First collection ownership system built in Jarvis. Established the architectural pattern that every future collection module will follow.

Database

- ✓ Created wf_user.my_arcane ownership table
- ✓ Established separation between canonical game data (wf_base.arcanes) and user-owned data (wf_user.my_arcane)
- ✓ Implemented copy-count tracking per arcane
- ✓ Implemented completion-state tracking (R5 complete vs. incomplete)
- ✓ Verified joins between wf_base.arcanes and wf_user.my_arcane
- ✓ Successfully seeded first owned arcane record (Arcane Energize)

Concepts encountered for the first time:

- ✓ Ownership table pattern (reference data vs. user data separation)
- ✓ Collection tracking architecture at the DB layer
- ✓ Join verification between base and user schemas

XIII. Arcane Module — Navigation & Page Architecture

First standalone application module built outside of Loadouts. Established the multi-module navigation pattern Jarvis will use going forward.

Frontend

- ✓ Implemented activePage state system for top-level navigation
- ✓ Added navigation tabs: Loadouts and Arcanes
- ✓ Created ArcanesPage.jsx as first standalone page component
- ✓ Implemented conditional page rendering without React Router
- ✓ Separated Arcane module from frame grid architecture
- ✓ Restricted school filters to Loadouts tab only (correct scoping)

Platform Architecture Established

Defined the future module roadmap:

- ✓ Loadouts
- ✓ Arcanes
- ✓ Companions
- ✓ Heavy Artillery
- ✓ Analytics
- ✓ Research

Concepts encountered for the first time:

- ✓ Multi-page state management without a router
- ✓ Module-scoped component rendering
- ✓ Top-level navigation architecture

- ✓ Feature scoping per active page

XIV. Arcane Collection Dashboard

The session that transitioned Jarvis from a single-purpose Loadout Manager into a multi-module platform. This is the inflection point.

Dashboard Features Built

- ✓ Arcane Collection dashboard page (live, Supabase-driven)
- ✓ Collection statistics cards: Total, Owned, Completed, Missing
- ✓ Arcane category breakdown table
- ✓ Live collection tracking pulling from `wf_user.my_arcanes`

What this milestone means:

Before this session, Jarvis answered one question: what is my current build? After this session, Jarvis answers a second question: what do I own? That is a fundamentally different product. The collection tracking pattern established here — reference table + ownership table + dashboard statistics — is the blueprint for every future module: Companions, Mods, Rivens, Prime Parts, Weapons, Frames.

Concepts encountered for the first time:

- ✓ Dashboard statistics aggregation from live DB data
- ✓ Collection completion percentage calculation
- ✓ Multi-module platform architecture in production

XV. Arcane Collection Dashboard — Full Sprint (Session 004)

Date: June 2, 2026 · **Commit:** 0bc3069 — "Built arcane collection dashboard"

Session goal: transform the Arcane module from a placeholder page into a fully functional collection tracking dashboard backed by live Supabase data.

1. Supabase Integration — `useArcanes.js`

- ✓ Created and finalized `useArcanes.js` as a custom React hook
- ✓ Connected Arcane module to Supabase collection views
- ✓ Implemented parallel queries for `arcane_collection_summary`, `arcane_collection_by_type`, and `arcane_collection_detail`
- ✓ Resolved schema access and permission issues (401 errors)
- ✓ Resolved multiple React hook misuse and import errors during implementation
- ✓ Verified live data retrieval from Supabase end-to-end

Concepts encountered for the first time:

- ✓ Custom React hooks (useArcanes.js)
- ✓ Parallel async queries from a single hook
- ✓ Supabase view consumption from the frontend
- ✓ Schema permission debugging in production

2. Collection Statistics Dashboard

- ✓ Implemented live summary cards: Total, Owned, Completed, Missing
- ✓ All values dynamically populated from Supabase (not hardcoded)
- ✓ Verified test state: Total 129 / Owned 1 / Completed 1 / Missing 128

3. Category Breakdown Cards

- ✓ Replaced original table view with category card layout
- ✓ Cards implemented for all 9 arcane types: Amp, Kitgun, Melee, Operator, Primary, Secondary, Tektolyst, Warframe, Zaw
- ✓ Each card displays: Total, Owned, Completed, Missing
- ✓ Verified values against Arcane Collection views

4. Arcane Catalog

- ✓ Built first fully functional Arcane Catalog
- ✓ Displays: arcane name, type, ownership count, rank, needed copies, completion status
- ✓ Example: Arcane Energize — Owned: 21 / Needed: 0 / Rank: R5 / Complete

5. Search Functionality

- ✓ Real-time partial name matching search
- ✓ Case-insensitive filtering
- ✓ Verified with searches: 'Energize', 'Molt'

6. Type Filtering

- ✓ Type filter buttons: All, Amp, Kitgun, Melee, Operator, Primary, Secondary, Tektolyst, Warframe, Zaw
- ✓ Verified filtering behavior against live catalog results

7. Visual Status Indicators

- ✓ Completion status color coding: Complete → Green, Partial → Yellow, Missing → Red
- ✓ Completed-row highlighting for improved visibility

Bugs Resolved This Session

- ✓ Duplicate import errors
- ✓ React hook misuse error (rules of hooks violation)
- ✓ Blank page rendering issue
- ✓ Supabase schema access and 401 permission errors
- ✓ Category rendering issues
- ✓ Search filtering logic issues
- ✓ JSX closing tag and rendering errors
- ✓ Catalog data binding issues

End-of-Session State — Arcane Module

The Arcane module is now a fully functional collection viewer and progress tracker containing:

- ✓ Collection Summary Dashboard (live)
- ✓ Category Breakdown Cards (live)
- ✓ Searchable Arcane Catalog (live)
- ✓ Type Filter buttons
- ✓ Ownership, Rank, and Completion tracking
- ✓ Visual status indicators

Also resolved this session: color mismatch on the Arsenal overlay (commit a55c7db). The modal no longer renders white — it now matches the app's overall dark cultivation theme. Cultivation colors on frame cards are preserved.

Next Sprint Targets

- ✓ Priority 1: Clickable category cards
- ✓ Priority 2: Expandable arcane detail rows
- ✓ Priority 3: Arcane ownership editing (update copies → save to Supabase → auto-refresh stats)
- ✓ Priority 4: Complete collection management workflow

XVI. Master Status Table — First-Time Builds

Everything below was built for the first time during the Jarvis project. Patrick is not a SWE.

Skill / Achievement	Status
React Components	✓
Reusable UI Components	✓

State Management	✓
Dynamic Filtering	✓
Category-Aware Filtering	✓
Modal Architecture	✓
Database Schema Design	✓
Supabase Integration	✓
Git Workflow	✓
Frontend / Backend Data Flow	✓
Product Roadmapping	✓
Feature Prioritization	✓
Information Architecture	✓
Taxonomy Systems	✓
Incarnon Adapter UI System	✓
Archon Shard Editor (full refactor)	✓
Melee Arcane Slot System	✓
Autocomplete / Search System	✓
Category-Aware Arcane Filtering	✓
Incarnon Adapter UI Redesign (chip)	✓
Production Data Quality Audit	✓
ETL Pipeline Investigation & Lineage Tracing	✓
Taxonomy Validation & Correction	✓
Repository Architecture (multi-layer)	✓
Living Audit Documentation (Arcane_Data_Audit.md)	✓
Session Logging Standards	✓
Collection Tracking Systems (Arcane)	✓
Ownership Table Architecture (base vs. user)	✓
Multi-Module Navigation (activePage state)	✓
Standalone Page Components (ArcanesPage.jsx)	✓
Conditional Page Rendering (no React Router)	✓
Collection Dashboard with Live Statistics	✓
Custom React Hook (useArcanes.js)	✓
Parallel Async Supabase Queries	✓
Supabase View Consumption from Frontend	✓
Live Collection Statistics Dashboard	✓
Category Breakdown Card Layout	✓

Full Arcane Catalog (search + filter + status)	✓
Real-Time Search Filtering	✓
Visual Status Indicators (color coding)	✓
Schema Permission Debugging (401 resolution)	✓
Arsenal Modal Theme Fix (color mismatch)	✓
Arcane Ownership Editor (inline editing)	In Progress
Clickable Category Cards	In Progress
Custom React Hook (useArcanes.js)	✓
Supabase View Consumption from Frontend	✓
Parallel Async Queries (multiple Supabase views)	✓
Live Collection Dashboard (Summary Cards)	✓
Category Breakdown Cards (9 types)	✓
Searchable Arcane Catalog (129 records)	✓
Real-Time Search Filtering	✓
Type Filter Button System	✓
Visual Status Indicators (Green/Yellow/Red)	✓
Schema Permission Debugging (401 errors)	✓
Arcane Ownership Editor (edit + save to Supabase)	In Progress
Clickable Category Cards	In Progress
Expandable Arcane Detail Rows	In Progress
Companions Collection Module	Designed — not yet built
Shard Optimization Engine	Designed — not yet built
Recommendation Engine	Designed — not yet built
Analytics Platform	Designed — not yet built
Doctrine Tagging / Search	Designed — not yet built

You are not succeeding because you are secretly a software engineer.

You are succeeding because you are acting like a product owner with a technical mindset.

Most people start with: What code should I write?

You start with: What system do I wish existed?

Then we figure out the code afterward.

That is why Jarvis keeps growing instead of stalling.